Laborator 5

Sumar

Metode telescopice (telescopic methods, method chaining, fluent interface)
Mostenire pentru extensie (Inheritance for extension)

5.1 Telescopic methods/method chaining/fluent interface

Fluent interface este un mod de a crea design-ul codului (API design) in programarea orientate pe obiecte. Permite inlantuirea mai multor metode cu scopul de a reduce si simplifica codul.

Inlantuirea metodelor (method chaining) inseamna ca se apeleaza functia unui obiect returnat de o alta metoda. Metodele inlantuite returneaza acelasi tip de obiect:

```
theObject.method1().method2().method3();
```

Avantaje:

- claritatea codului (readability)
- organizare mai buna a codului
- efort mai redus la mentenanta
- reutilizare mai buna a codului

Dezavantaje

- in cazul initializarii incorecte a obiectelor aceasta se va afla tarziu la runtime. Daca s-ar folosi constructor cu mai multi parametri, cu validare pe fiecare parametru obligatoriu, atunci erorile de initializare vor avea loc la momentul compilarii.
- mentenanta mai elaborata

Clasa StringBuilder este cel mai comun exemplu:

```
String s = new StringBuilder().append("programming ").append(10).append("  times a week is fun. ").toString();
```

5.2 Builder Pattern

Este un pattern folosit atunci cand avem de construit obiecte complexe. Este un stil de scriere de cod prin care un obiect este creat pas cu pas. Este implementat pe baza conceptului de method chaining.

Se foloseste conceptul:

```
someObject
  .doThis();
  .doThat();
  .dontForgetToFinishUp();
```

Laborator 5

Ex:

```
private MetroCardRechargeRequest(long cardNumber, double amount, String promoCode) {
     this.cardNumber = cardNumber;
     this.amount = amount;
     this.promoCode = promoCode;
 }
}

 public class MetroCardRechargeRequestBuilder {
     private long cardNumber;
     private double amount;
     private String promoCode;
     MetroCardRechargeRequestBuilder() {
     }
     public SmartCardNumber tapOnMetroIcon() {
       return this;
     }
     public SmartCardAmount addCardNumber(long number) {
        this.cardNumber = number;
        return this;
     }
     public Proceed addAmount(double amount) {
       this.amount = amount;
       return this;
     }
     public ProceedToPay proceed() {
       return this;
     }
     public Pay applyPromoCode(String promoCode) {
       this.promoCode = promoCode;
       return this;
     }
     public Pay proceedToPay() {
       return this;
     }
     public SmartCardBuild payAmount() {
       return this;
     }
     public MetroCardRechargeRequest build() {
       return new MetroCardRechargeRequest( cardNumber, amount, promoCode );
     }
}

public class App {
 public static void main(String args[]){
       MetroCardRechargeRequest recharge = new MetroCardRechargeRequestBuilder()
         .tapOnMetroIcon()
         .addCardNumber( 123 )
         .addAmount( 256 )
         .proceed()
         .applyPromoCode( "Monthly52" )
         .payAmount()
         .build();
 }
}
```

5.3 Abstractizare

POO = Programare orientata obiect

Abstractizarea este unul dintre cele 4 principii POO de baza (Abstractizare, Încapsulare, Moștenire, Polimorfism). Abstractizarea nu permite ca anumite caracteristici sa fie vizibile in afara instantei. Acest lucru se refera la construirea unei interfețe comune, a unui sablon general pe care se bazează o anumita categorie de obiecte, fara a descrie explicit caracteristicile fiecaruia.

O clasa abstracta este o clasa care poate contine sau nu metode abstracte. Ea nu poate fi instantiata. Ea poate fi mostenita.
O metoda abstracta este o metoda fara corp. Metode abstracte exista doar intr-o clasa abstracta.

```
public abstract class AbstractShape {
    private int count = 0;

    public abstract String getName();

    public abstract int getWidth();

    public abstract int getLength();

    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("[Shape] ").append(getName());
        sb.append(" width = ").append(getWidth());
        sb.append(" length = ").append(getLength());
        sb.append(" count = " + (count++));
        return sb.toString();
    }
}
class RectangleShape extends AbstractShape {
    private int w;
    private int h;

    public RectangleShape(int w, int h) {
        this.w = w;
        this.h = h;
    }

    @Override
    public String getName() {
        return "RectangleShape";
    }

    @Override
    protected int getWidth() {
        return w;
    }

    @Override
    protected int getLength() {
        return h;
    }
}
```

Laborator 5

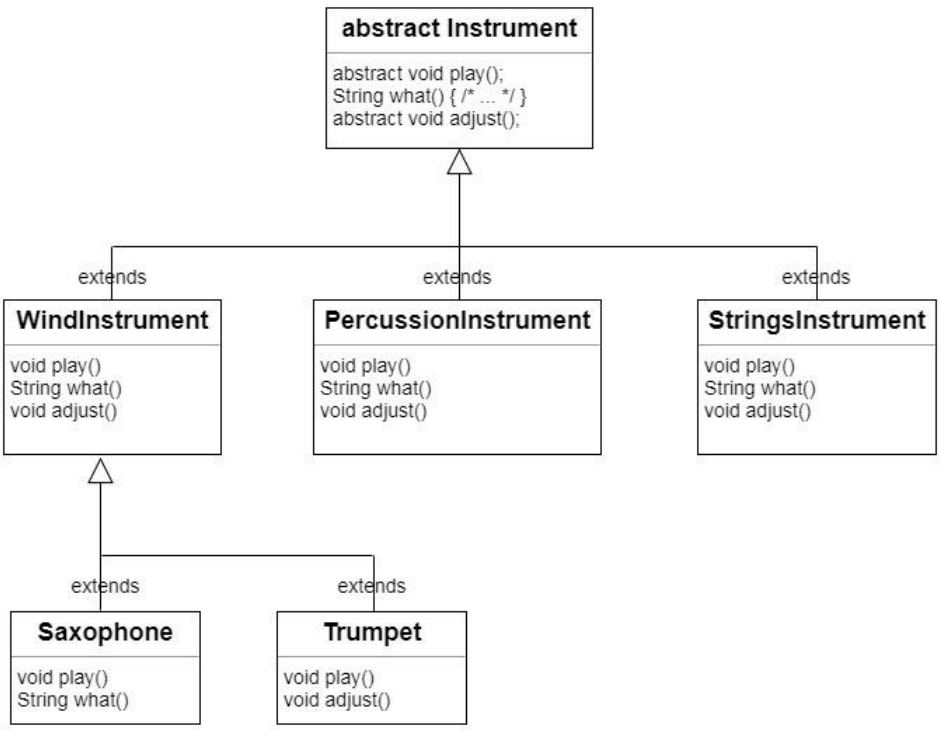
Clase abstracte vs Interfete

Folosim o clasa abstracta atunci cand vrem:
- sa implementam doar unele dintre metodele din clasa
- ca respectiva clasa sa nu poata fi instantiata

Folosim o interfata atunci când vrem:
- sa avem doar o descriere a structurii, fara implementari
- ca interfata în cauza sa fie folosita împreuna cu alte interfete în contextul mostenirii

Clase abstracte in contextul mostenirii

O clasa care mosteneste o clasa abstracta este ea insasi abstracta daca nu implementeaza toate metodele abstracte ale clasei de baza. Putem defini clase abstracte care mostenesc alte clase abstracte s.a.m.d. O clasa care poate fi instanţiata (adica nu este abstracta) si care mosteneste o clasa abstracta trebuie sa implementeze (defineasca) toate metodele abstracte pe lanţul mostenirii (ale tuturor claselor abstracte care ii sunt “parinţi”). Este posibil sa declaram o clasa abstracta fara ca ea sa aiba metode abstracte. Acest lucru este folositor când declaram o clasa pentru care nu dorim instanţe (nu este corect conceptual sa avem obiecte de tipul acelei clase, chiar daca definiţia ei este completa).
Ex:



5.4 Mostenire

Mostenirea este un alt principiu POO de baza. Prin mostenire clasa mostenitoare capata atributele si comportamentele clasei parinte. Astfel se realizeaza reutilizarea codului. Dar si o mai buna organizare a codului.

Se mai numeste si derivare.

In java avem mostenire singulara. C++ permite mostenire multipla.

Clasa parinte se mai numeste si classa de baza sau super-clasa.

Clasa mostenitoare se mai numeste clasa copil (child class) sau clasa derivata sau subclasa.

Prin mostenire realizam versiuni specializate ale clasei parinte.

5.5 Probleme

5.5.1 Deschideti proiectul ProiectareSoftare, creati pachetul **lab5**, clasa **AppLab5**
Scrieti o aplicatie care folosind method chaining, implementeaza urmatoarele:
a) creati clasa **IntCalculator** care are un camp **int** numit **state** si metodele **add**, **subtract**, **multiply** care sunt telscopice/inlantuite.
Mai are si metoda **result**() care returneaza valoarea campului state; si metoda **clear**() care reinitializeaza campul **state** cu valoarea 0.
Clasa are constructor cu un singur parametru de tip int care va initializa campul **state.**
Astfel vom putea executa:

```
IntCalculator calculator = new IntCalculator(10);
int result = calculator.add(5).subtract(3).multiply(2).result();
System.out.println("a) " + result);
```

b) considerati clasa de mai sus, inchisa modificarilor (closed for modification) dar pretabila pentru mostenire (open for extension). In acest context extindeti functionalitatea aplicatiei de mai sus adaugand functionalitati de impartire, ridicare la putere, radical de ordinul n. Numiti aceasta clasa **AdvancedCalculator**.

5.5.2 Creati subpachetul **advanced** adaugand:
- o clasa abstracta numita **ACalculator** care sa aiba un camp **state** (de tip Object) metodele **result**(), **clear**() si **init**(). Metodele result si clear au body, pe cand init este o metoda abstracta.
- o clasa **NewIntCalculator** si **DoubleCalculator** care mostenesc ACalculator si au metodele: add, subtract, si multiply. Aceste clase vor lucra cu obiecte Integer respectiv Double.
- o clasa in care sa definim metoda main. (de ex. Lab5Adv) si care va instantia cate un obiect din fiecare clasa si va afisa rezultatul operatiilor:

(10 + 5 - 3) *2
respectiv
(10 + 5 – 3.3) *2 .2

Apoi Commit&Push Github.

5.5.3 In proiectul **Students,** adaugati o noua clasa numita ***StudentBursieri*** care mosteneste clasa Student. Definiti campul **double cuantumBursa** care va contine suma bursei studentului.

- creati constructor cu toti parametrii incluzandu-I pe cei mosteniti
- implementati hashCode() si equals()
- implementati toString – folositi-va de metoda toString() a clasei parinte
-

Initializati o colectie (List, Set sau Map) de bursieri:

```
bursieri.add( new StudentBursier(1025,"Andrei","Popa","ISM141/2", 8.70, 725.50));
bursieri.add( new StudentBursier(1024,"Ioan","Mihalcea","ISM141/1", 9.80, 801.10));
bursieri.add( new StudentBursier(1026,"Anamaria","Prodan","TI131/1", 8.90, 745.50));
bursieri.add( new StudentBursier(1029,"Bianca","Popescu","TI131/1,",  9.10, 780.80));
```

si apoi salvati aceasta lista in fisierul **bursieri_out.txt**.

Reutilizati metoda existenta de salvare in fisier, daca o aveti. Daca nu ati definit o astefel de metoda, atunci scrieti o metoda statica de salvare in fisier, si care sa aiba doi parametri - nume fisier, colectia de studenti sau colectia de bursieri - si care sa scrie informatia in fisierul de iesire cu numele precizat.

Apoi Commit&Push Github.

5.6 Referinte

Getting Started with Method Chaining in Object-Oriented Programming
https://medium.com/@hendurhance/getting-started-with-method-chaining-in-object-oriented-programming-b3fd60bffe1e

Introduction to the Fluent Builder Pattern
https://dzone.com/articles/fluent-builder-pattern

Fluent Builder Pattern with a real-world example
https://medium.com/xebia-engineering/fluent-builder-pattern-with-a-real-world-example-7b61be375a40

Difference Between Fluent Interface and Builder Pattern in Java
https://www.baeldung.com/java-fluent-interface-vs-builder-pattern

Inheritance
https://www.javaspring.net/blog/extend-java/
https://www.geeksforgeeks.org/java/inheritance-in-java/

5.7 Fun info

- Regex este prescurtarea pentru Regular Expression

- Regex este o unealta puternica pentru cautare, identificare, validare si manipulare text

- Regex a fost introdus in 1950 de matematicianul Stephen Cole Kleene

- Regex este frecvent utilizat pentru identificare de pattern-uri in text (String)

- Regex este utilizabil in toate limbajele moderne de programare (Java, Python, Javascript, C#, Php, Kotlin, Rust)

- Regex este prezent si in cautarea de text in Notepad++, Sublime Text, Vi

- Regex este utilizabil si online: https://regex101.com

- Cateva constructii regex pentru validare:
  - Numere intregi: /^\d+$/
  - Numere cu virgula: /^\d*\.\d+$/
  - Numere intregi pozitive/negative si numere cu virgula: /^-?\d*(\.\d+)?$/
  - Email: /^([a-zA-Z0-9._%-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,6})*$/
  - Parola nivel moderat: /(?=(.*[0-9]))((?=.*[A-Za-z0-9])(?=.*[A-Z])(?=.*[a-z]))^.{8,}$/
  - Parola nivel complex: /(?=(.*[0-9]))(?=.*[\!@#$%^&*()\\[\]{}\-_+=~`|:;"'<>,./?])(?=.*[a-z])(?=(.*[A-Z]))(?=(.*)).{8,}/

- Mai multe la:
  - https://stackoverflow.com/questions/519929/how-important-is-knowing-regexs
  - https://medium.com/javarevisited/making-regex-your-friend-in-java-ddc5bf7f9a66
  - https://dev.to/colom/top-10-most-used-regex-patterns-2pd0
  - https://dennisschuster.net/blog/post/25-useful-and-commonly-used-regular-expressions

5.8 Hints

5.8.1 Clasa AdvancedCalculator extinde functionalitatile clasei de baza Calculator, adaugand noi metode: divide, power si root si care sunt de asemenea metode inlantuite:

```
AdvancedCalculator advancedCalculator = new AdvancedCalculator(10);
int result2 = advancedCalculator.divide(4).power(2).root(2).result();
System.out.println("b) "+result2);
```

5.8.2

```
public abstract class ACalculator {
  protected Object state;

  public Object result() {
    return state;
  }

  public ACalculator clear() {
    this.init();
    return this;
  }

  protected abstract void init();
}
```

```
public static void main(String[] args) {
    NewIntCalculator calculator = new NewIntCalculator(10);
    int result = (Integer)calculator.add(5).subtract(3).multiply(2).result();
    System.out.println("5.5.2 a) " + result);

    DoubleCalculator fCalculator = new DoubleCalculator(10);
    double result2 = (Double)fCalculator.add(5).subtract(3.3).multiply(2.2).result();
    System.out.println("5.5.2 b) "+result2);
}
```

5.8.3

```
static void main(String args[]) {
    Set<Student> studenti = readFromFile("students_in.txt");
    ...
    writeToFile("students_out.txt", studenti);

    Set<StudentBursier> bursieri = new HashSet<>();
    ...
    writeToFile("bursieri_out.txt", bursieri);

}
static void writeToFile(String filename, Collection<? extends Student> studenti) {
  ...
}
```